16 luglio 2019

Politecnico di Milano

Anno accademico 2018-2019

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Cugola	☐ Margara	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, smartwatch, ...).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si consideri la seguente specifica della classe `BinaryTree` che modella un albero binario di elementi (non nulli) di tipo `T`.

```
public class BinaryTree<T> {
  // Costruisce un albero binario con un solo elemento nella sua radice.
  public BinaryTree(T rootElem) ;

  // Ritorna l'elemento alla radice dell'albero
  public /*@ pure @*/ T getRoot() ;

  // Ritorna un Iterator che itera esattamente una volta su ogni elemento nell'albero.
  public /*@ pure @*/ Iterator<T> allElements() ;

  // Inserisce l'elemento elem come figlio sinistro del nodo che contiene l'elemento
  // father, purchè tale figlio sinistro non esista già, in tal caso solleva
  // l'eccezione NonEmptyChildException. Se father non è parte dell'albero,
  // solleva l'eccezione NoSuchElementException. Se l'elemento elem è già
  // parte dell'albero solleva l'eccezione ExistingElementException.
  public void insertLeft(T father, T elem) throws NonEmptyChildException,
    NoSuchElementException, ExistingElementException ;

  // Inserisce l'elemento elem come figlio destro del nodo che contiene l'elemento
  // father, purchè tale figlio destro non esista già, in tal caso solleva
  // l'eccezione NonEmptyChildException. Se father non è parte dell'albero,
  // solleva l'eccezione NoSuchElementException. Se l'elemento elem è già
  // parte dell'albero solleva l'eccezione ExistingElementException.
  public void insertRight(T father, T elem) throws NonEmptyChildException,
    NoSuchElementException, ExistingElementException ;

  // Ritorna true se l'elemento elem è parte dell'albero, falso altrimenti.
  public /*@ pure @*/ boolean contains(T elem) ;

  // Ritorna l'elemento a sinistra dell'elemento father (null se tale figlio
  // sinistro non esiste). Solleva l'eccezione NoSuchElementException se father
  // non è parte dell'albero.
  public /*@ pure @*/ T left(T father) throws NoSuchElementException ;

  // Ritorna l'elemento a destra dell'elemento father (null se tale figlio
  // destro non esiste). Solleva l'eccezione NoSuchElementException se father
  // non è parte dell'albero.
  public /*@ pure @*/ T right(T father) throws NoSuchElementException ;

  // Ritorna l'elemento padre dell'elemento elem; null se elem è la radice
  // dell'albero. Solleva l'eccezione NoSuchElementException se elem
  // non è parte dell'albero.
  public /*@ pure @*/ T father(T elem) throws NoSuchElementException ;
}
```

Domanda a)

Si specifichino in JML i metodi `insertLeft` e `father`.

Soluzione

Definiamo:

```
unchanged(): getRoot().equals(\old(getRoot()) &&
             (\forall T el; ; contains(el) <==> \old(contains(el))) &&
             (\forall T el; contains(el) ;
                 \old(left(el)==null) ? left(el)==null : left(el).equals(\old(left(el)))) &&
```

```
            (\forall T el; contains(el) ;
                \old(right(el)==null) ? right(el)==null : right(el).equals(\old(right(el)))) &&
            (\forall T el; contains(el) ;
                getRoot().equals(el) ? father(el) == null : father(el).equals(\old(father(el))))
```

Usiamo quindi `unchanged` per la nostra specifica:

```
//@requires father != null && elem != null
//@ensures \old(contains(father)) && \old(!contains(elem)) && \old(left(father)==null) &&
//@ left(father).equals(elem) && getRoot().equals(\old(getRoot())) &&
//@ (\forall T el; \old(contains(el)) ; contains(el)) &&
//@ (\forall T el; !el.equals(elem) && contains(el) ; \old(contains(el))) &&
//@ (\forall T el; !el.equals(father) && \old(contains(el)) ;
//@   \old(left(el)==null) ? left(el)==null : left(el).equals(\old(left(el)))) &&
//@ (\forall T el; contains(el) ;
//@   \old(right(el)==null) ? right(el)==null : right(el).equals(\old(right(el)))) &&
//@ (\forall T el; \old(contains(el)) ;
//@   getRoot().equals(el) ? father(el) == null : father(el).equals(\old(father(el))))
//@ left(elem)==null && right(elem)==null
//@signals (NonEmptyChildException e) \old(left(father)!=null) && unchanged();
//@signals (NoSuchElementException e) \old(!contains(father)) &&  unchanged();
//@signals (ExistingElementException e) \old(contains(el) && unchanged();
public void insertLeft(T father, T elem) throws NonEmptyChildException,
  NoSuchElementException, ExistingElementException ;

//@requires elem != null
//@ensures \old(contains(elem) && (elem.equals(getRoot()) => \result==null) &&
//@ (!elem.equals(getRoot()) => ( left(\result).equals(elem) || right(\result).equals(elem)))
//@signals (NoSuchElementException e) \old(!contains(elem))
public /*@ pure @*/ T father(T elem) throws NoSuchElementException ;
```

Domanda b)

Si consideri una versione `BinaryTreeBis` della classe `BinaryTree` la cui unica differenza è che l'iteratore restituisce gli elementi in ordine di inserzione. La classe `BinaryTreeBis` può essere definita come erede di `BinaryTree` rispettando il principio di sostituzione di Liskov?

Soluzione

Si, visto che l'iteratore di `BinaryTree` non precisa un ordine particolare.

Domanda c)

Si fornisca l'invariante pubblico della classe `BinaryTree`.

Soluzione

```
//@public invariant
//@ getRoot()!=null && contains(getRoot()) && !contains(null) &&
//@ (\forall T el; contains(el) && left(el)!=null; contains(left(el))) &&
//@ (\forall T el; contains(el) && right(el)!=null; contains(right(el)))
//@ (\forall T el; contains(el);
//@        getRoot().equals(el) ? father(el) == null :
//@                               father(el) != null && contains(father(el))) &&
//@ (\forall T el; contains(el) && left(el)!=null ; father(left(el)).equals(el)) &&
//@ (\forall T el; contains(el) && right(el)!=null ; father(right(el)).equals(el)) &&
//@ (\forall T el; contains(el);
//@   !left(el).equals(getRoot()) && !right(el).equals(getRoot() &&
//@ (\forall T el1; contains(el1); (\forall T el2; contains(el2) && !el1.equals(el2);
//@   !left(el1).equals(left(el2)) && !left(el1).equals(right(el2)) &&
//@   !right(el1).equals(left(el2)) && !right(el1).equals(right(el2))))
```

Esercizio 2

Si consideri la seguente classe `Numbers` che implementa una sequenza di numeri reali di dimensioni fissata al momento della costruzione. Il metodo `average` calcola la media dei due elementi in posizione `pos1` e `pos2` impostando a tale valor medio entrambi gli elementi.

```
public class Numbers {
  private double data[];
  private Object lock[];

  public Numbers(int size) {
    data = new double[size];
    lock = new Object[size];
    for (int i=0; i<size; i++) lock[i] = new Object();
  }

  public void set(int pos, double val) {
    synchronized(lock[pos]) { data[pos] = val; }
  }

  public double get(int pos) {
    synchronized(lock[pos]) { return data[pos]; }
  }

  public void average(int pos1, int pos2) {
    synchronized(lock[pos1]) {
      synchronized(lock[pos2]) {
        double avg = (data[pos1] + data[pos2]) / 2.0;
        data[pos1] = data[pos2] = avg;
      }
    }
  }
}
```

Domanda a)

La classe `Numbers` presenta un problema di sincronizzazione potendo dar luogo a deadlock. Si scriva una sequenza di chiamate da parte di due thread paralleli che può portare a un deadlock e si indichi come modificare il codice della classe `Numbers` per evitare il rischio di deadlock pur mantenendo una sincronizzazione che eviti conflitti nell'accesso ai dati.

Soluzione

Il deadlock deriva dalla doppia sincronizzazione del metodo `average`. È sufficiente che un thread invochi `average(i,j)` e l'altro `average(j,i)` per i medesimi valori di `i` e `j` perché il deadlock possa manifestarsi.

Una possibile soluzione consiste nell'eliminare la sincronizzazione con oggetti espliciti (attributo `lock`) sincronizzando tutti i metodi su `this`.

Domanda b)

Sulla base della versione corretta dela classe `Numbers` elaborata al punto precedente, si aggiunga un metodo `waitForPositiveSum()` che sospende il chiamante fino a quando la somma di tutti gli elementi in `Numbers` è negativa. Si devono modificare altri metodi della classe `Numbers`?

Soluzione

Si riporta nel seguito una versione corretta della classe `Numbers` che elimina il rischio di deadlock e aggiunge il metodo `waitForPositiveSum()`. Si noti l'aggiunta dell'istruzione `notifyAll` al termine del metodo `set`.

```
public class NumbersOk {
  private double data[];

  public NumbersOk(int size) {
    data = new double[size];
  }

  public synchronized void set(int pos, double val) {
    data[pos] = val;
    notifyAll();
  }

  public synchronized double get(int pos) {
    return data[pos];
  }

  public synchronized void average(int pos1, int pos2) {
    double avg = (data[pos1] + data[pos2]) / 2.0;
    data[pos1] = data[pos2] = avg;
  }

  public synchronized void waitForPositiveSum() throws InterruptedException {
    double sum;
    do {
      sum = 0.0;
      for (int i=0; i<data.length; i++) sum += data[i];
      if (sum<0.0) wait();
    } while (sum>0.0);
  }
}
```

Esercizio 3

Un framework per lo sviluppo di applicazioni mobili integra un sistema di localizzazione `LocationService` che deve essere istanziato diversamente a seconda dell'hardware installato sullo specifico device. Al momento, esistono tre implementazioni concrete: `GPSLocationService` che sfrutta GPS, `WiFiLocationService` che sfrutta WiFi, e `DummyLocationService` utilizzato in ambiente di test (quando non sono presenti GPS e WiFi). Una classe `Device` ritorna informazioni sulla specifica dotazione hardware del dispositivo in uso. Le definizioni delle classi sono riportate di seguito.

```
public interface LocationService {
   public Location getLocation();
}

public class GPSLocationService implements LocationService {
   @Override
   Location getLocation() { ... }
}

public class WiFiLocationService implements LocationService {
   @Override
   Location getLocation() { ... }
}

public class DummyLocationService implements LocationService {
```

```
   @Override
   Location getLocation() { ... }
}

public class Device {
   public static boolean hasGPS() { ... }
   public static boolean hasWiFi() { ... }
}
```

Domanda a)

Mediante l'uso di un apposito design pattern, progettare una soluzione che nasconda agli utilizzatori la scelta della specifica implementazione di `LocationService` e semplifichi l'evoluzione del sistema nel caso nuove implementazioni di `LocationService` diventino disponibili in futuro.

Soluzione

Si può ricorrere a un factory method. La logica di creazione è in questo modo localizzata in un solo punto del codice e può essere modificata/estesa facilmente in futuro.

```
public class LocationServiceFactory {
   public static LocationService getLocationService() {
      if (Device.hasGPS()) return new GPSLocationService();
      else if (Device.hasWiFi()) return new WiFiLocationService();
      else return new DummyLocationService();
   }
}

// Utilizzo
LocationService locationService = LocationServiceFactory.getLocationService();
```

Domanda b)

Un'applicazione di chat sfrutta il `LocationService` per gestire richieste di invio posizione ai contatti. La concreta modalità di gestione delle richieste dipende dallo stato di privacy in cui si trova l'applicazione. Attualmente sono disponibili due stati di privacy, `High` e `Low`.

Mediante l'uso di un apposito design pattern, progettare una soluzione che gestisca gli stati di privacy e semplifichi l'evoluzione del sistema nel caso nuovi stati di privacy diventino disponibili in futuro.

Si completi il codice di `ChatApp` (anche aggiungendo opportuni attributi e metodi) e si fornisca la definizione di eventuali classi che si ritiene necessario aggiungere.

```
public class ChatApp {
   ...
   public void handleLocationRequest(LocationService service) { ... }
   ...
}
```

Soluzione

Si può ricorrere a uno state pattern. La logica di gestione delle richieste viene delegata a un oggetto `PrivacyState`. Future estensioni possono avvenire mediante l'implementazione di nuove sotto-classi di `PrivacyState`, senza modificare il codice di `ChatApp`.

```
public interface PrivacyState {
   public void handleRequest(LocationService service);
}

public class HighPrivacy implements PrivacyState {
   @Override
```

```
   public void handleRequest(LocationService service) { ... }
}

public class LowPrivacy implements PrivacyState {
   @Override
   public void handleRequest(LocationService service) { ... }
}

public class ChatApp {
   private PrivacyState privacyState;

   public ChatApp(PrivacyState privacyState) {
      this.privacyState = privacyState;
   }

   public void handleLocationRequest(LocationService service) {
      privacyState.handleRequest(service);
   }

   public void setPrivacyState(PrivacyState privacyState) {
      this.privacyState = privacyState;
   }
}
```

Esercizio 4

Si consideri il seguente metodo Java:

```
public static boolean m(Predicate<String> p, String s) {
  if (p == null) {
    p = x -> true;
  }
  if (s == null) {
    s = "";
  }
  if (s.length() < 2) return false;
  else if (p.test(s)) return true;
  else return false;
}
```

Domanda a)

Si fornisca un insieme di casi di test per la copertura delle istruzioni (statement coverage).

Soluzione

Un singolo caso permette di entrare nei tre rami `if` ed eseguire le istruzioni in essi contenute:

`p = null, s = null`.

Servono poi un caso di test per entrare nel ramo `else if` e un caso di test per entrare nel ramo `else`, rispettivamente:

`p = x -> true, s = "aaa"`

`p = x -> false, s = "aaa"`

Domanda b)

Si fornisca un insieme di casi di test per la copertura delle decisioni (edge coverage).

Soluzione

I casi precedenti sono sufficienti per coprire anche le decisioni.